

EC2x 系列&EG25-G

QuecOpen 蓝牙 API 参考手册

LTE Standard 模块系列

版本：1.0

日期：2021-02-24

状态：受控文件



上海移远通信技术股份有限公司始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司

上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233

电话：+86 21 51086236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，可随时登陆如下网址：

<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

上海移远通信技术股份有限公司提供该文档内容用以支持其客户的产品设计。客户须按照文档中提供的规范、参数来设计其产品。因未能遵守有关操作或设计规范而造成的损害，上海移远通信技术股份有限公司不承担任何责任。在未声明前，上海移远通信技术股份有限公司有权对该文档进行更新。

免责声明

上海移远通信技术股份有限公司尽力确保开发中功能的完整性、准确性、及时性或效用，但不排除上述功能错误或遗漏的可能。除非其他有效协议另有规定，否则上海移远通信技术股份有限公司对开发中功能的使用不做任何暗示或明示的保证。在适用法律允许的最大范围内，上海移远通信技术股份有限公司不对任何因使用开发中功能而遭受的损失或损害承担责任，无论此类损失或损害是否可以预见。

保密义务

除非上海移远通信技术股份有限公司特别授权，否则我司所提供文档和信息的接收方须对接收的文档和信息保密，不得将其用于除本项目的实施与开展以外的任何其他目的。未经上海移远通信技术股份有限公司书面同意，不得获取、使用或向第三方泄露我司所提供的文档和信息。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，上海移远通信技术股份有限公司有权追究法律责任。

版权申明

本文档版权属于上海移远通信技术股份有限公司，任何人未经我司允许而复制转载该文档将承担法律责任。

版权所有 ©上海移远通信技术股份有限公司 2021，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2021.

文档历史

修订记录

版本	日期	作者	变更表述
-	2021-01-05	Arthur CHEN	文档创建
1.0	2021-02-24	Arthur CHEN	受控版本

目录

文档历史	2
目录	3
图片索引	5
表格索引	6
1 引言	7
1.1. 适用模块	7
1.2. 特殊符号	8
2 蓝牙配置流程图	9
2.1. BLE 配置流程图	9
2.2. SPP 配置流程图	10
2.3. AG 配置流程图	11
3 蓝牙接口函数	12
3.1. BLE 接口函数	12
3.1.1. ql_ble_power_on	12
3.1.2. ql_ble_power_off	12
3.1.3. ql_ble_client_init	13
3.1.4. ql_ble_client_deinit	13
3.1.5. ql_ble_set_local_name	14
3.1.6. ql_ble_gatt_register	14
3.1.7. ql_ble_gatt_unregister	14
3.1.8. ql_ble_db_service_add	15
3.1.9. ql_ble_db_service_del	15
3.1.10. ql_ble_db_charact_add	16
3.1.11. ql_ble_db_charact_del	17
3.1.12. ql_ble_db_descriptor_add	17
3.1.13. ql_ble_db_descriptor_del	18
3.1.14. ql_ble_gatt_send_indication	19
3.1.15. ql_ble_gatt_send_notification	20
3.1.16. ql_ble_gatt_read_response	20
3.1.17. ql_ble_gatt_write_response	21
3.1.18. ql_ble_adverting_start	22
3.1.19. ql_ble_adverting_stop	22
3.1.20. ql_ble_gatt_set_adverting_param	23
3.1.21. ql_ble_gatt_disconnect	23
3.1.22. ql_ble_gatt_peripheral	24
3.1.23. ql_ble_gatt_db_alloc	24
3.1.24. ql_ble_gatt_db_dealloc	25
3.1.25. ql_ble_db_service_set_active	25
3.1.26. ql_ble_gatt_db_add	26
3.1.27. ql_ble_gatt_db_remove	26

3.1.28.	ql_ble_db_show	27
3.2.	SPP 接口函数	27
3.2.1.	ql_spp_power_on	27
3.2.2.	ql_spp_power_off	28
3.2.3.	ql_spp_client_init	28
3.2.4.	ql_activate_spp	28
3.2.5.	ql_deactivate_spp	29
3.2.6.	ql_connect_spp	29
3.2.7.	ql_write_spp	30
3.2.8.	ql_disconnect_spp	30
3.3.	AG 接口函数	31
3.3.1.	ql_hfg_power_on	31
3.3.2.	ql_ble_power_off	31
3.3.3.	ql_ble_client_init	31
3.3.4.	ql_open_scan_device	31
3.3.5.	ql_close_scan_device	32
3.3.6.	ql_hfg_connect	32
3.3.7.	ql_hfg_cancelconnect	33
4	蓝牙使用演示步骤	34
4.1.	BLE 配置演示步骤	34
4.2.	SPP 配置演示步骤	35
4.2.1.	模块为从设备	35
4.2.2.	模块为主设备	36
4.3.	AG 配置演示步骤	36
5	附录 参考文档和术语缩写	37

图片索引

图 1: BLE 配置流程图..... 9

图 2: SPP 配置流程图 10

图 3: AG 配置流程图11

表格索引

表 1: 适用模块 7

表 2: 特殊符号 8

表 3: 参考文档 37

表 4: 术语缩写 37

1 引言

移远通信 LTE Standard EC2x 系列和 EG25-G 模块支持 QuecOpen®方案；QuecOpen® 是开源的基于 Linux 的嵌入式开发平台，可简化 IoT 应用的软件设计和开发过程。有关 QuecOpen®的详细信息，请参考文档 [1]。

本文档主要介绍了移远通信 LTE Standard EC2x 系列和 EG25-G 模块的蓝牙功能，该功能需与移远通信 FC20 系列或 FC21 模块结合使用，以极低的功耗通过无线技术实现设备互连。

蓝牙（BT）技术是一种无线数据和语音通信开放的全球规范，是基于低成本的近距离无线连接，为固定设备和移动设备建立通信环境的一种特殊的近距离无线技术连接。蓝牙技术包括传统蓝牙和低功耗蓝牙。本文档将重点介绍蓝牙协议栈中 BLE、SPP 和 HFP 功能在移远通信 EC2x 系列和 EG25-G 模块上的实现。

蓝牙低功耗（Bluetooth Low Energy，或称 Bluetooth LE、BLE）也称低功耗蓝牙，是蓝牙技术联盟设计和销售的一种个人局域网技术，旨在用于医疗保健、运动健身、信标、安防、家庭娱乐等领域的新兴应用。相较经典蓝牙，低功耗蓝牙旨在保持同等通信范围的同时显著降低功耗和成本。

传统蓝牙的开发基于 SPP 协议，能在本地蓝牙设备和远端蓝牙设备之间建立一条传输通道，实现数据的交互。

HFP 在蓝牙协议栈中控制蓝牙设备拨打电话，如接听、挂断、语音和拒接等。HFP 定义了音频网关角色（AG）和免提组件角色（HF*）两个角色：HF 角色为音频网关的远程音频输入输出的机制，并提供若干遥控功能，一般用作车载蓝牙；AG 角色为音频设备的输入输出网关，一般用于手机端。目前仅对 AG 角色相关内容进行介绍。

1.1. 适用模块

表 1: 适用模块

模块系列	模块
EC2x 系列	EC25 系列
	EC21 系列
	EC20 R2.1

EG25-G

EG25-G

1.2. 特殊符号

表 2: 特殊符号

符号	定义
*	若无特别说明，模块功能、特性、接口、引脚名称、AT 命令或参数后面所标记的星号（*）表示该功能、特性、接口、引脚名称、AT 命令或参数正在开发中，因此暂不支持。

2 蓝牙配置流程图

2.1. BLE 配置流程图

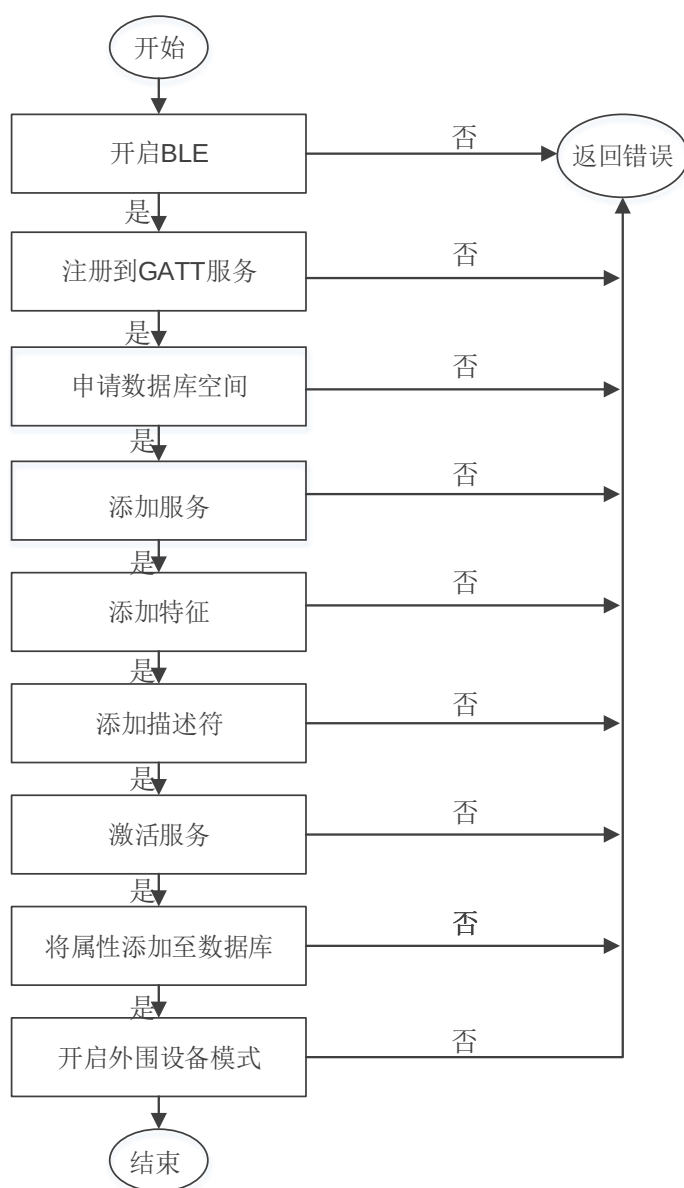


图 1: BLE 配置流程图

2.2. SPP 配置流程图

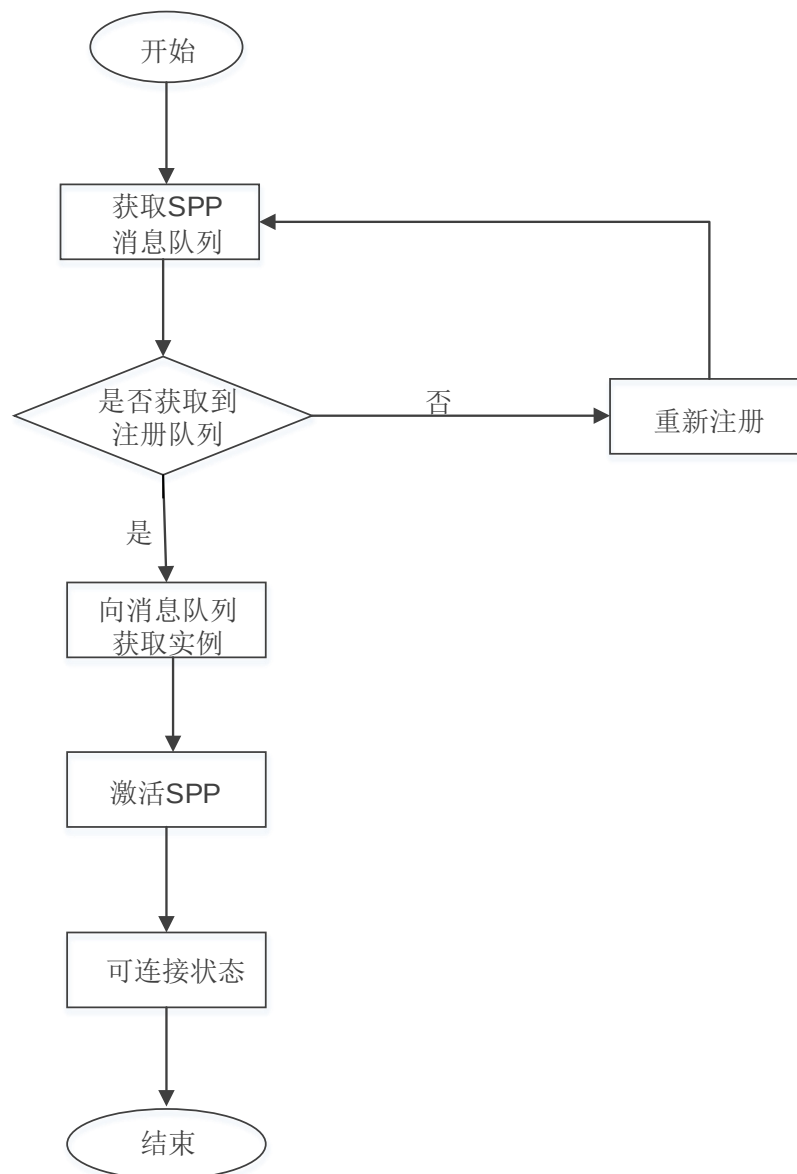


图 2: SPP 配置流程图

2.3. AG 配置流程图

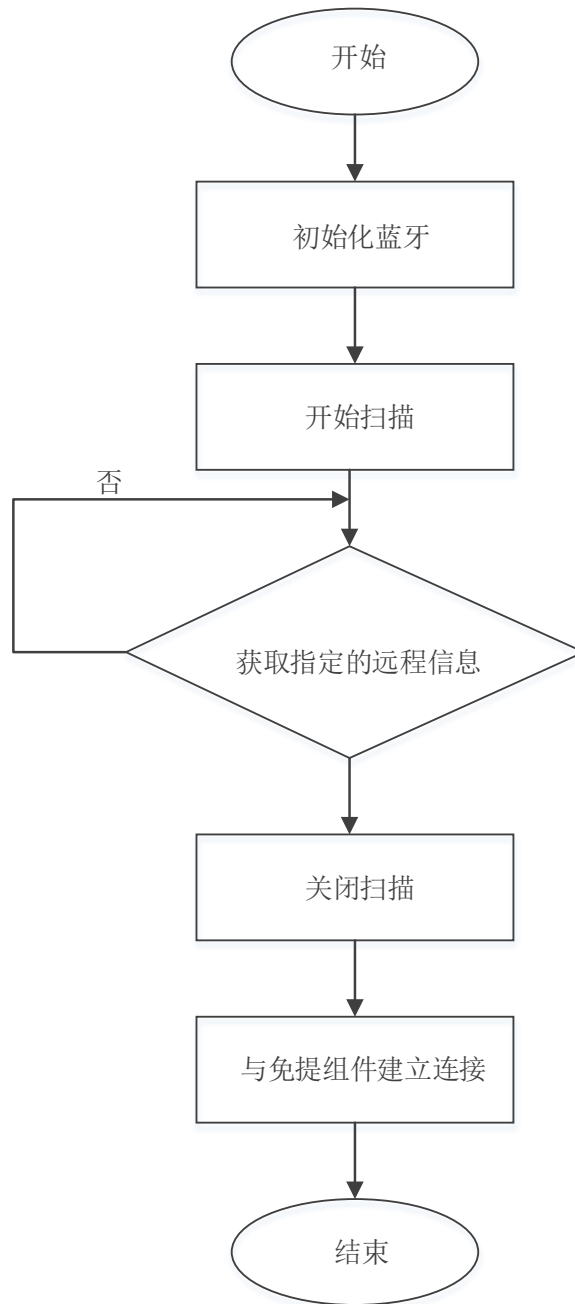


图 3: AG 配置流程图

3 蓝牙接口函数

3.1. BLE 接口函数

3.1.1. ql_ble_power_on

该函数用于开启 BLE。

- 函数原型

```
int ql_ble_power_on();
```

- 参数

无

- 返回值

0	执行成功
其他值	执行失败

3.1.2. ql_ble_power_off

该函数用于关闭 BLE。

- 函数原型

```
int ql_ble_power_off();
```

- 参数

无

- 返回值

0	执行成功
其他值	执行失败

3.1.3. ql_ble_client_init

配置 GATT 之前，该函数用于注册用户回调接口。

● 函数原型

```
int ql_ble_client_init(int (*client_cb)(QuecBtPrim type, char *data, int len));
```

● 参数

client_cb:

[In] 处理服务端发送的事件并给予应答。

type:

[In] 接收的事件类型。

data:

[In] 接收的数据。

len:

[In] 数据长度。

● 返回值

0 执行成功

其他值 执行失败

3.1.4. ql_ble_client_deinit

该函数用于去初始化客户端环境。

● 函数原型

```
int ql_ble_client_deinit();
```

● 参数

无

● 返回值

0 执行成功

其他值 执行失败

3.1.5. ql_ble_set_local_name

该函数用于设置设备名称。

- 函数原型

```
int ql_ble_set_local_name(char *name);
```

- 参数

name:

[In] 设备名称。

- 返回值

0 执行成功

其他值 执行失败

3.1.6. ql_ble_gatt_register

该函数用于注册 GATT 服务，获取 GATT ID。

- 函数原型

```
int ql_ble_gatt_register(QuecBtGattId *gattId);
```

- 参数

gattId:

[Out] 已注册的 GATT ID。

- 返回值

0 执行成功

其他值 执行失败

3.1.7. ql_ble_gatt_unregister

该函数用于注销 GATT 服务。

- 函数原型

```
int ql_ble_gatt_unregister(QuecBtGattId *gattId);
```

- 参数

gattId:

[In] 需要注销的 GATT ID。

- 返回值

0 执行成功

其他值 执行失败

3.1.8. ql_ble_db_service_add

该函数用于在 GATT 服务上添加服务。

- 函数原型

```
int ql_ble_db_service_add(QuecBtGattId gattId, QuecBtUint16 svrID, QuecBtUuid16 uuid,
QuecBtUint8 isPrimary);
```

- 参数

gattId:

[In] GATT ID。

svrID:

[In] 需要添加的服务 ID。

uuid:

[In] 服务 UUID。

isPrimary:

[In] 标识服务为主要服务或次要服务或被其它服务项所引用的服务。

- 返回值

0 执行成功

其他值 执行失败

3.1.9. ql_ble_db_service_del

该函数用于在 GATT 服务上删除已添加的服务。

- 函数原型

```
int ql_ble_db_service_del(QuecBtGattId gattId, QuecBtUint16 svrID,);
```

- 参数

gattId:

[In] GATT ID。

svrID:

[In] 需要删除的服务 ID。

- 返回值

0 执行成功

其他值 执行失败

3.1.10. ql_ble_db_charact_add

该函数用于添加特征。

- 函数原型

```
int ql_ble_db_charact_add(QuecBtGattId gattId, QuecBtUInt16 svrID, QuecBtUInt16 charactID,
QuecBtUInt16 uuid, QuecBtUInt16 valueLength, QuecBtUInt8 prop, QuecBtUInt16 attrValueFlags,
QuecBtUInt8 *value);
```

- 参数

gattId:

[In] GATT ID。

svrID:

[In] 服务 ID。

charactID:

[In] 需添加的特征 ID。

uuid:

[In] 特征 UUID。

valueLength:

[In] 特征的长度。

prop:

[In] 特征的性质。不同的值代表不同的性质。

attrValueFlags:

[In] 属性值标识。定义访问特征值的方式。

value:

[In] 特征的值。

- 返回值

0 执行成功

其他值 执行失败

3.1.11. ql_ble_db_charact_del

该函数用于删除已添加的特征。

- 函数原型

```
int ql_ble_db_charact_del(QuecBtGattId gattId, QuecBtUint16 svrID, QuecBtUint16 charactID);
```

- 参数

gattId:

[In] GATT ID。

svrID:

[In] 服务 ID。

charactID:

[IN] 需删除的特征 ID。

- 返回值

0 执行成功

其他值 执行失败

3.1.12. ql_ble_db_descriptor_add

该函数用于添加描述符。

- 函数原型

```
int ql_ble_db_descriptor_add(QuecBtGattId gattId, QuecBtUint16 svrID, QuecBtUint16 charactID,
QuecBtUint16 descID, QuecBtUuid16 uuid, QuecBtUint16 valueLength, QuecBtUint8 prop,
QuecBtUint16 attrValueFlags, QuecBtUint8 *value);
```

- 参数

gattId:

[In] GATT ID。

svrID:

[In] 服务 ID。

charactID:

[In] 特征 ID。

descID:

[In] 需要添加的描述符的 ID。

uuid:

[In] 描述符 UUID。

valueLength:

[In] 描述符的长度。

prop:

[In] 描述符的性质。不同的值代表不同的性质。

attrValueFlags:

[In] 属性值标识。定义访问特征值的方式。

value:

[In] 描述符的值。

● 返回值

0	执行成功
其他值	执行失败

3.1.13. ql_ble_db_descriptor_del

该函数用于删除已添加的描述符。

● 函数原型

```
int ql_ble_db_descriptor_del(QuecBtGattId gattId, QuecBtUInt16 svrID, QuecBtUInt16 charactID,
QuecBtUInt16 descID);
```

● 参数

gattId:

[In] GATT ID。

svrID:

[In] 服务 ID。

charactID:

[In] 删除的特征 ID。

descID:

[In] 需要删除的描述符的 ID。

● 返回值

0 执行成功

其他值 执行失败

3.1.14. ql_ble_gatt_send_indication

该函数用于发送指示。

● 函数原型

```
int ql_ble_gatt_send_indication(QuecBtGattId gattId, QuecBtConnId connId, QuecBtUint16 attrHandle, QuecBtUint16 valueLength, QuecBtUint8 *value);
```

● 参数

gattId:

[In] GATT ID。

connId:

[In] 已建立 BLE 连接的 ID。

attrHandle:

[In] 属性句柄。

valueLength:

[In] 指示的长度。

value:

[In] 指示的内容。

● 返回值

0 执行成功

其他值 执行失败

3.1.15. ql_ble_gatt_send_notification

该函数用于发送通知。

- 函数原型

```
int ql_ble_gatt_send_notification(QuecBtGattId gattId, QuecBtConnId connId, QuecBtUint16 attrHandle, QuecBtUint16 valueLength, QuecBtUint8 *value);
```

- 参数

gattId:

[In] GATT ID。

connId:

[In] 已建立 BLE 连接的 ID。

attrHandle:

[In] 属性句柄。

valueLength:

[In] 通知的长度。

value:

[In] 通知的内容。

- 返回值

0 执行成功

其他值 执行失败

3.1.16. ql_ble_gatt_read_response

该函数用于对数据读取作出响应。

- 函数原型

```
int ql_ble_gatt_read_response(QuecBtGattId gattId, QuecBtConnId connId, QuecBtUint16 attrHandle, QuecResultCode result, QuecBtUint16 valueLength, QuecBtUint8* value);
```

- 参数

gattId:

[In] GATT ID。

connId:

[In] 已建立 BLE 连接的 ID。

attrHandle:

[In] 属性句柄。

result:

[In] 返回值。

valueLength:

[In] 返回值的长度。

value:

[In] 响应内容。

● 返回值

0 执行成功

其他值 执行失败

3.1.17. ql_ble_gatt_write_response

该函数用于对写入的数据做出响应。

● 函数原型

```
int ql_ble_gatt_write_response(QuecBtGattId gattId, QuecBtConnId connId, QuecBtUint16 attrHandle, QuecResultCode result);
```

● 参数

gattId:

[In] GATT ID。

connId:

[In] 已建立 BLE 连接的 ID。

attrHandle:

[In] 属性句柄。

result:

[In] 返回值。

- 返回值

0	执行成功
其他值	执行失败

3.1.18. ql_ble_adverting_start

该函数用于激活广播。

- 函数原型

```
int ql_ble_adverting_start(QuecBtGattId gattId);
```

- 参数

gattId:
[In] GATT ID。

- 返回值

0	执行成功
其他值	执行失败

3.1.19. ql_ble_adverting_stop

该函数用于终止广播。

- 函数原型

```
int ql_ble_adverting_stop(QuecBtGattId gattId);
```

- 参数

gattId:
[In] GATT ID。

- 返回值

0	执行成功
其他值	执行失败

3.1.20. ql_ble_gatt_set_advertising_param

该函数用于设置广播间隔。

- 函数原型

```
int ql_ble_gatt_set_advertising_param(QuecBtGattId gattId, QuecBtUint16 advIntervalMin,
QuecBtUint16 advIntervalMax);
```

- 参数

gattId:

[In] GATT ID。

advIntervalMin:

[In] 最小广播间隔。范围：32~16384；单位：0.625 毫秒；默认值：256。

advIntervalMax:

[In] 最大广播间隔。范围：32~16384；单位：0.625 毫秒；默认值：512。

- 返回值

0 执行成功

其他值 执行失败

3.1.21. ql_ble_gatt_disconnect

该函数用于断开与 GATT 服务的连接。

- 函数原型

```
int ql_ble_gatt_disconnect(QuecBtGattId gattId, QuecBtConnId connId);
```

- 参数

gattId:

[In] GATT ID。

connId:

[In] 已建立 BLE 连接的 ID。

- 返回值

0 执行成功

其他值 执行失败

3.1.22. ql_ble_gatt_peripheral

该函数用于开启外围设备模式。

- 函数原型

```
int ql_ble_gatt_peripheral(QuecBtGattId gattId, QuecBtTypedDeviceAddr addr,
QuecBtGattConnFlags flags, QuecBtUint16 mtu, QuecBtConnId *connId);
```

- 参数

gattId:

[In] GATT ID。

addr:

[In] 设备地址。

flags:

[In] 连接标识。

mtu:

[In] 建立连接期间通知远程设备的最大传输单位。

connId:

[In] 已建立连接的 ID。

- 返回值

0 执行成功

其他值 执行失败

3.1.23. ql_ble_gatt_db_alloc

该函数用于申请数据库空间。

- 函数原型

```
int ql_ble_gatt_db_alloc (QuecBtGattId gattId, QuecBtUint16 numOfAttrHandles, QuecBtUint16
preferredStartHandle);
```

- 参数

gattId:

[In] GATT ID。

numOfAttrHandles:

[In] 属性数量。

preferredStartHandle:

[In] 优先使用的句柄。

- 返回值

0 执行成功

其他值 执行失败

3.1.24. ql_ble_gatt_db_dealloc

该函数用于释放数据库空间。

- 函数原型

```
int ql_ble_gatt_db_dealloc (QuecBtGattId gattId);
```

- 参数

gattId:

[In] GATT ID。

- 返回值

0 执行成功

其他值 执行失败

3.1.25. ql_ble_db_service_set_active

该函数用于激活数据库中的服务。

- 函数原型

```
int ql_ble_db_service_set_active(QuecBtGattId gattId, QuecBtUint16 svrID, QuecBtUint8 isActive);
```

- 参数

gattId:

[In] GATT ID。

svrID:

[In] 服务 ID。

isActive:

[In] 激活状态。

- 返回值

0 执行成功

其他值 执行失败

3.1.26. ql_ble_gatt_db_add

该函数用于将服务添加到数据库。

- 函数原型

```
int ql_ble_gatt_db_add(QuecBtGattId gattId);
```

- 参数

gattId:

[In] GATT ID。

- 返回值

0 执行成功

其他值 执行失败

3.1.27. ql_ble_gatt_db_remove

该函数用于删除申请的数据库。

- 函数原型

```
int ql_ble_gatt_db_remove(QuecBtGattId gattId);
```

- 参数

gattId:

[In] GATT ID。

- 返回值

0 执行成功

其他值 执行失败

3.1.28. ql_ble_db_show

该函数用于在标准输出中打印数据库信息。

- 函数原型

```
int ql_ble_db_show(QuecBtGattId gattId);
```

- 参数

gattId:
[In] GATT ID。

- 返回值

0	执行成功
其他值	执行失败

3.2. SPP 接口函数

3.2.1. ql_spp_power_on

该函数用于开启蓝牙服务器。

- 函数原型

```
int ql_spp_power_on();
```

- 参数

无

- 返回值

0	执行成功
其他值	执行失败

3.2.2. ql_spp_power_off

该函数用于关闭蓝牙服务器。

- 函数原型

```
int ql_spp_power_off()
```

- 参数

无

- 返回值

0	执行成功
其他值	执行失败

3.2.3. ql_spp_client_init

该函数用于初始化 SPP 客户端。

- 函数原型

```
int ql_spp_client_init();
```

- 参数

无

- 返回值

0	执行成功
其他值	执行失败

3.2.4. ql_activate_spp

该函数用于将移远通信 FC20 系列或 FC21 模块激活为 SPP 从设备并等待终端设备连接。

- 函数原型

```
int ql_activate_spp();
```

- 参数

无

- 返回值

0 执行成功

其他值 执行失败

3.2.5. ql_deactivate_spp

该函数用于去激活 SPP。

- 函数原型

```
int ql_deactivate_spp();
```

- 参数

无

- 返回值

0 执行成功

其他值 执行失败

3.2.6. ql_connect_spp

该函数用于将 SPP 激活为主设备并连接其他 SPP 设备。

- 函数原型

```
int ql_connect_spp(QuecDeviceAddr * addr);
```

- 参数

addr:

[In] 从设备 MAC 地址

- 返回值

0 执行成功

其他值 执行失败

3.2.7. ql_write_spp

该函数用于向远程 SPP 设备发送数据。

- 函数原型

```
int ql_write_spp(QuecBtUint16 valueLength, QuecBtUint8 * value);
```

- 参数

valueLength:

[In] 将要发送的数据长度

value:

[In] 将要发送的数据

- 返回值

0 执行成功

其他值 执行失败

3.2.8. ql_disconnect_spp

该函数用于主动断开 SPP 连接。

- 函数原型

```
int ql_disconnect_spp();
```

- 参数

无

- 返回值

0 执行成功

其他值 执行失败

3.3. AG 接口函数

3.3.1. ql_hfg_power_on

该函数用于开启蓝牙服务器。

- 函数原型

```
int ql_hfg_power_on();
```

- 参数

无

- 返回值

0 执行成功

其他值 执行失败

3.3.2. ql_ble_power_off

该函数用于关闭蓝牙服务器。详情请参考第3.1.2章。

- 函数原型

```
int ql_ble_power_off()
```

3.3.3. ql_ble_client_init

该函数用于初始化 BLE 为 AG 设备。详情请参考第3.1.3章。

- 函数原型

```
int ql_ble_client_init(int (*client_cb)(QuecBtPrim type, char *data, int len))
```

3.3.4. ql_open_scan_device

该函数用于开启蓝牙设备扫描。

- 函数原型

```
int ql_open_scan_device();
```

- 参数

无

- 返回值

0 执行成功
其他值 执行失败

3.3.5. ql_close_scan_device

该函数用于关闭蓝牙设备扫描。

- 函数原型

```
int ql_close_scan_device();
```

- 参数

无

- 返回值

0 执行成功
其他值 执行失败

3.3.6. ql_hfg_connect

该函数用于与免提组件建立连接。

- 函数原型

```
int ql_hfg_connect(QuecBtDeviceAddr address);
```

- 参数

address:
[In] 免提组件的 MAC 地址。

- 返回值

0 执行成功
其他值 执行失败

3.3.7. ql_hfg_cancelconnect

该函数用于断开与免提组件的连接。

- 函数原型

```
int ql_hfg_cancelconnect(QuecBtDeviceAddr address) ;
```

- 参数

address:

[In] 免提组件的 MAC 地址。

- 返回值

0 执行成功

其他值 执行失败

4 蓝牙使用演示步骤

4.1. BLE 配置演示步骤

步骤1: 执行如下脚本，开启 BLE。

```
int ql_ble_power_on();
```

步骤2: 调用如下函数对客户端进行初始化。其中回调函数 *client_cb* 为用户自行注册，用于对服务端发送的报文进行对应解析。

```
ql_ble_client_init(client_cb);
```

步骤3: 调用如下函数注册 GATT 服务以获取一个 GATT ID，请将注册后的 ID 存放至 *gattId* 指向的地址空间中。

```
ql_ble_gatt_register(&gattId);
```

步骤4: 调用如下函数申请一段数据库空间，该空间用于存放属性的描述符。

```
ql_ble_gatt_db_alloc(gattId, numOfAttrHandles, preferredStartHandle);
```

步骤5: 调用如下函数添加服务，随后可在该服务基础上添加特征与描述符。

```
ql_ble_db_service_add(gattId, 1, QUEC_BT_GATT_UUID_DEVICE_INFORMATION_SERVICE, 1);
```

步骤6: 调用如下函数在服务基础上添加特征。

```
ql_ble_db_charact_add(gattId,1,1,
QUEC_BT_GATT_UUID_MANUFACTURER_NAME_STRING_CHARAC,128,
QUEC_ATT_PERM_READ | QUEC_ATT_PERM_INDICATE,
QUEC_BT_GATT_ATTR_FLAGS_IRQ_READ, name);
```

步骤7: 调用如下函数在服务及其特征的基础上添加描述符。

```
ql_ble_db_descriptor_add(gattId, 1, 1, 1, 0x2902, 2, QUEC_ATT_PERM_READ,
QUEC_BT_GATT_ATTR_FLAGS_NONE, (QuecBtUint8 *)&value);
```

步骤8: 调用如下函数激活服务，将上述添加的服务一起注册到服务端。

```
ql_ble_db_service_set_active(gattId, 1, 1);
```

步骤9: 调用如下函数将服务添加到数据库，即添加至申请的数据库空间。

```
ql_ble_gatt_db_add(gattId);
```

步骤10: 调用如下函数设置为外围设备模式，等待建立连接。

```
ql_ble_gatt_peripheral(gattId, t_addr, QUEC_BT_GATT_FLAGS_UNDIRECTED, 0, &connId);
```

4.2. SPP 配置演示步骤

4.2.1. 模块为从设备

步骤1: 调用如下函数开启蓝牙服务器。

```
ql_spp_power_on();
```

步骤2: 调用如下函数对 SPP 客户端进行初始化。

```
ql_spp_client_init();
```

步骤3: 调用如下函数将 SPP 激活为从设备。

```
ql_activate_spp();
```

4.2.2. 模块为主设备

步骤1: 调用如下函数开启蓝牙服务器。

```
ql_spp_power_on();
```

步骤2: 调用如下函数对 SPP 客户端进行初始化。

```
ql_spp_client_init();
```

步骤3: 调用如下函数将 SPP 激活为主设备并连接至 SPP 从设备。

```
ql_connect_spp();
```

4.3. AG 配置演示步骤

步骤1: 调用如下函数开启蓝牙服务器。

```
ql_hfg_power_on();
```

步骤2: 调用如下函数初始化 BLE 为 AG 设备。

```
ql_ble_client_init(client_cb_func);
```

步骤3: 调用如下函数扫描蓝牙设备。

```
ql_open_scan_device();
```

步骤4: 调用如下函数停止扫描。

```
ql_close_scan_device();
```

步骤5: 调用如下函数与免提组件建立连接。

```
ql_hfg_connect(remote_device_addr);
```

5 附录 参考文档和术语缩写

表 3: 参考文档

序号	文档名称	描述
[1]	Quectel_EC2x&EG9x&EG25-G 系列 _QuecOpen_快速开发指导	EC2x&EG9x&EG25-G 系列 QuecOpen 方案的 快速开发指导

表 4: 术语缩写

缩写	英文全称	中文全称
AG	Audio Gateway	音频网关
API	Application Programming Interface	应用程序编程接口
BLE	Bluetooth Low Energy	蓝牙低功耗
BT	Bluetooth	蓝牙
GATT	Generic Attribute Profile	通用属性规范
HF	Hands Free	免提组件
HFP	Hands-free Profile	免提网关协议
ID	Identity	识别号
IoT	Internet of Things	物联网
LTE	Long-Term Evolution	长期演进
SPP	Serial Port Profile	蓝牙串口
UUID	Universally Unique Identifier	通用唯一识别码